

1. Introduction

Assignment 7 extends the Assignment 6 console Rock-Paper-Scissors implementation into a Qt6 GUI application. The core objective is to support both text mode and GUI mode in one codebase without duplicating game logic.

This report focuses on the required deliverables: inversion of control in GUI mode, screenshots, reuse and refactoring from Assignment 6, and run instructions for both modes.

2. Requirements and Design Goals

The assignment specifies two classes of requirements: functional behavior and software design quality.

The functional GUI requirements are:

1. Allow users to choose the computer strategy (random or smart).
2. Allow users to set the number of game rounds.
3. Allow users to enter the human move each round.
4. Display current round number.
5. Display the human move for the round.
6. Display the computer prediction of the human move.
7. Display the computer move.
8. Display round winner (or tie).
9. Display cumulative wins and ties after each round.

The architectural goals were:

1. Keep one application executable in either text mode or GUI mode.
 2. Maximize reuse of Assignment 6 domain and strategy logic.
 3. Implement GUI behavior through event-driven IoC rather than procedural polling.
 4. Preserve extensibility so future modes can be added without major rewrites.
-

3. System Architecture

3.1 Layered decomposition

The implementation follows a layered decomposition with explicit responsibility boundaries.

1. Domain layer: defines Choice values, round outcome rules, and RoundResult payload.

2. Strategy layer: defines Strategy interface and concrete policies RandomStrategy and SmartStrategy.
3. Player layer: provides HumanPlayer, GUIHumanPlayer, and ComputerPlayer abstractions.
4. Engine layer: centralizes round progression and score bookkeeping in GameEngine.
5. Presentation layer: implements console output in ConsoleGameEngine and GUI rendering in MainWindow plus GUIGameEngine.
6. Persistence layer: stores learned frequencies in FrequencyStore.

This design prevents direct dependence of domain logic on UI widgets, which is essential for dual-mode execution.

3.2 Core abstractions

GameEngine defines the reusable game lifecycle and invariant operations such as applying round outcomes to cumulative scores. Both mode-specific engines derive from this common base. ConsoleGameEngine maps lifecycle callbacks to terminal output. GUIGameEngine maps lifecycle callbacks to Qt signals consumed by MainWindow.

The Strategy abstraction allows runtime substitution of move-selection behavior. ComputerPlayer delegates move generation and history updates to the active strategy, enabling policy replacement without changing engine code.

3.3 Architectural rationale

The selected architecture was guided by three principles.

1. Single responsibility: each class owns a narrow concern.
 2. Open/closed behavior: strategies and front-ends are replaceable by extension rather than modification.
 3. Interface-driven collaboration: high-level control flow depends on abstractions, not concrete widgets.
-

4. Inversion of Control in GUI Mode

Inversion of control (IoC) is the most important conceptual difference between console and GUI execution.

4.1 Control flow in text mode

Text mode uses a synchronous control loop:

1. Initialize game and strategy state.
2. For each round, read input from stdin.
3. Compute round result.
4. Print output.
5. Save learned data at completion.

In this model, application code owns the call sequence.

4.2 Control flow in GUI mode

In GUI mode, Qt owns the call sequence through its event loop:

1. Application creates MainWindow and starts QApplication event processing.
2. User actions trigger widget signals.
3. Connected slots in MainWindow invoke engine methods.
4. GUIGameEngine emits round and completion signals.
5. MainWindow updates labels, images, and score widgets.

The framework decides when callbacks execute. Application code provides handlers and state transitions only.

4.3 Signal-slot mediated lifecycle

The lifecycle is modeled as event transitions:

1. Start event initializes a game session and locks setup controls.
2. Choice event submits one human move and advances one round.
3. Round-result event updates round summary and cumulative metrics.
4. Game-finished event restores setup controls and displays final outcome.

This separation permits responsive UI updates without embedding game rules in UI callbacks.

4.4 Engineering consequences

Adopting IoC required explicit state management. Control enabling and disabling is used to guard invalid transitions, such as changing strategy during active play. This yields a safer state machine and reduces UI-level race conditions caused by inconsistent user actions.

In practice, GUI state is controlled by a simple lifecycle:

1. Idle state: setup controls enabled, move buttons disabled.
2. Active game state: setup controls disabled, move buttons enabled.
3. Completed state: move buttons disabled, setup controls re-enabled.

This lifecycle prevents illegal transitions, such as changing algorithm or round count in the middle of a game, and keeps engine/UI state synchronized.

Another IoC consequence is separation of concerns between producer and consumer of round events. GUIGameEngine produces structured result signals, while MainWindow consumes those signals to update labels, images, and score displays. This design keeps UI rendering code out of core game logic and improves maintainability.

5. Reuse from Assignment 6 and Refactoring Strategy

5.1 Reused components

The project intentionally preserved existing components from Assignment 6.

1. Round rule evaluation and Choice representation.
2. Strategy interface and core strategy implementations.
3. ComputerPlayer delegation model.
4. FrequencyStore file persistence.
5. Score accumulation logic in the engine core.

This reuse reduced implementation risk and maintained behavioral continuity between modes.

5.2 Required refactoring for dual-mode support

Supporting two interaction paradigms required targeted refactoring.

1. Engine flow was split into synchronous and event-driven entry points.
2. Display hooks were abstracted so console and GUI output paths remain independent.
3. Strategy diagnostics (prediction and confidence) were exposed for GUI visualization.
4. GUIHumanPlayer was introduced to bridge event input into the shared engine model.
5. Startup logic in main was expanded to parse mode-selection arguments.

5.3 Refactoring outcome

The refactor preserved a clean dependency direction. UI code depends on core logic, but core logic does not depend on Qt widgets. This is the key property enabling long-term extension to additional front-ends.

Refactoring also improved testability and change safety:

1. Round outcome logic remains centralized, so rule changes are made in one place.
2. Strategy replacement is localized to strategy construction, not engine flow logic.
3. Presentation changes in MainWindow do not require edits to GameEngine.
4. Console and GUI behavior stay consistent because both use the same underlying round processing pipeline.

5.4 Assignment 6 to Assignment 7 change log (design-level)

This subsection records the most significant architectural deltas from Assignment 6 to Assignment 7.

1. Startup behavior:
 - Assignment 6: text mode only startup path.
 - Assignment 7: mode-aware startup path supporting both GUI and text execution from one binary.
2. Engine flow model:

- Assignment 6: primarily synchronous playGame loop.
- Assignment 7: synchronous console loop plus event-oriented GUI progression via startGame and per-round callbacks.

3. Strategy observability:

- Assignment 6: strategy output consumed mostly as final move choice.
- Assignment 7: strategy prediction/confidence exposed for GUI transparency and educational feedback.

4. UI and domain separation:

- Assignment 6: no GUI layer.
- Assignment 7: MainWindow and GUIGameEngine added as adapters, without moving outcome rules into UI code.

5. User interaction model:

- Assignment 6: blocking terminal input via HumanPlayer.
- Assignment 7: event-driven interaction via GUIHumanPlayer and button-triggered handlers.

These changes were intentionally minimal with respect to domain rules and intentionally significant with respect to interaction architecture.

Implementation-wise, the most important refactoring decisions were:

1. Keep persistence boundaries unchanged (load before gameplay, save after gameplay), so Smart strategy behavior remains compatible with existing frequency data.
2. Add prediction/confidence accessors at the strategy interface level rather than hard-coding Smart-specific logic into the UI.
3. Introduce GUIHumanPlayer as an adapter so user input source changes without changing game rule processing.

6. Smart Strategy and Learning Model

6.1 Prediction model

The smart strategy uses sequence-frequency prediction over encoded move histories. Given a prefix of recent observations, the strategy estimates the most probable next human move by comparing stored continuation frequencies. The computer then plays the deterministic counter move.

6.2 Online update process

For each completed round:

1. Human move is encoded into a token sequence.
2. Rolling history is updated with fixed maximum length.
3. Once the sequence reaches configured length, the corresponding key frequency is incremented.

4. Updated frequencies are eventually persisted to disk.

This supports incremental online learning without requiring batch retraining.

6.3 Persistence behavior

FrequencyStore serializes key-count pairs to frequencies.dat. On startup, the strategy loads prior data, allowing performance to improve across sessions when user behavior is pattern-like.

6.4 Confidence signaling

When a matching prefix has no learned continuation, the strategy falls back to random play and reports zero confidence. When data exists, confidence equals observed frequency count for the selected continuation. Displaying confidence in the GUI improves interpretability of strategy decisions.

6.5 Practical behavior and interpretation

Against a highly random human, smart performance converges toward random baseline because no stable pattern can be exploited. Against repetitive users, smart mode tends to outperform random mode as frequency evidence accumulates. This distinction is expected and aligns with the design intent of a pattern-exploitation strategy.

7. GUI Design Rationale

7.1 Information hierarchy

The GUI places setup controls at the top, current-round state in the center, and history on a side panel. This layout keeps the active decision flow visible while preserving contextual history.

7.2 Interaction constraints

Control-state management prevents invalid transitions: move buttons stay disabled before start, setup controls are locked during active play, and controls are restored when a session ends.

7.3 Visual communication

The interface combines hand images, outcome text, score labels, and prediction feedback so each round can be interpreted quickly.

8. Execution Instructions

This section addresses the assignment requirement to document text-mode and GUI-mode execution.

8.1 Prerequisites

1. CMake version 3.16 or higher.
2. C++17 compiler toolchain.
3. Qt6 Widgets development libraries.

8.2 Build procedure

```
cmake -S . -B build
cmake --build build
```

8.3 GUI mode

Default GUI launch:

```
./build/rps
```

Explicit GUI launch:

```
./build/rps --gui
```

8.4 Text mode

Random strategy:

```
./build/rps --random --rounds=20
```

Smart strategy:

```
./build/rps --smart --rounds=20 --n=5
```

8.5 Suggested demonstration sequence

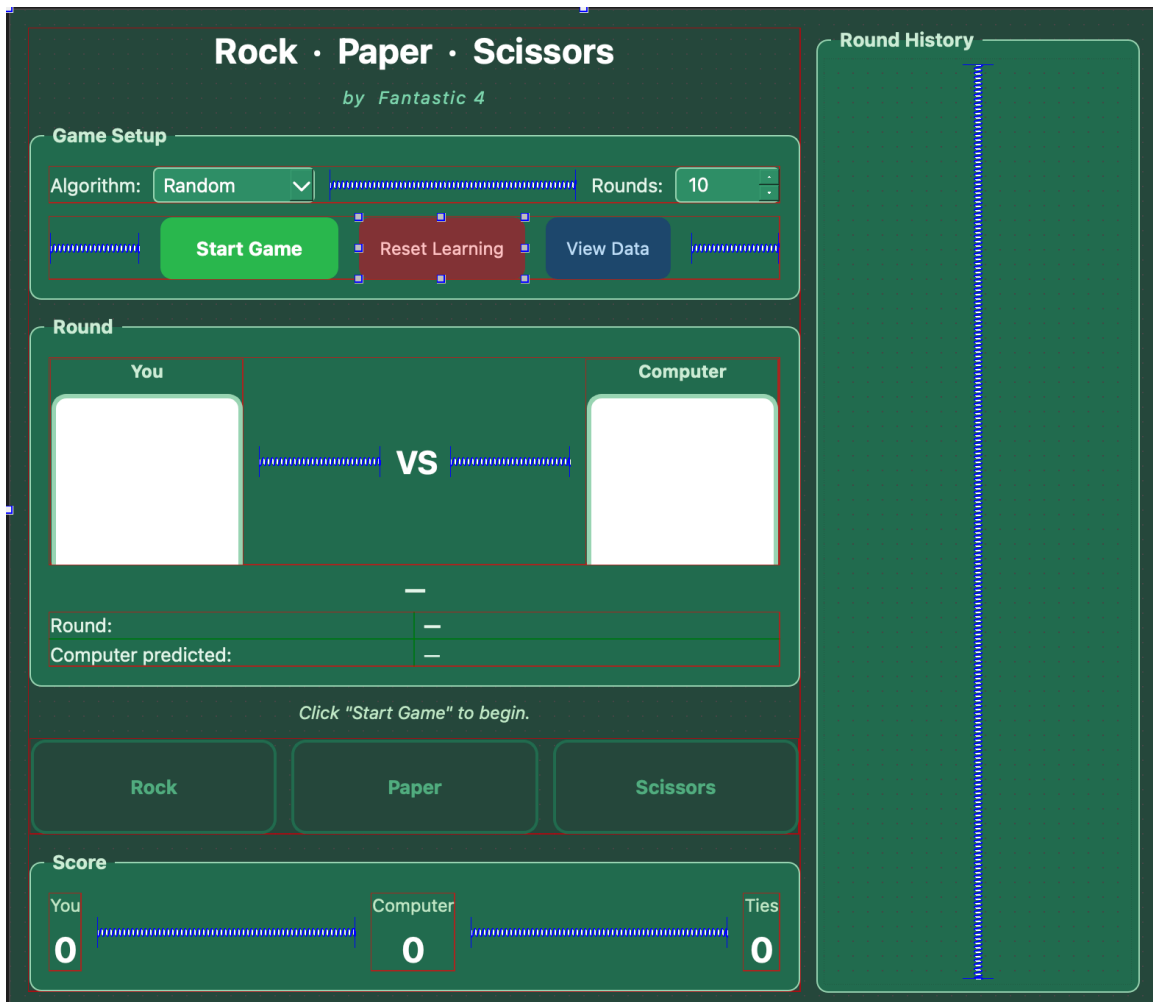
1. Run random text mode for a short baseline.
 2. Run smart mode with repeated human patterns.
 3. Launch GUI mode and verify prediction, outcome, and score updates.
-

9. Required Screenshots

Figure 1. Qt Designer Layout

Place this image file at:

- Assignment7/report-assets/fig1_qt_designer_layout.png

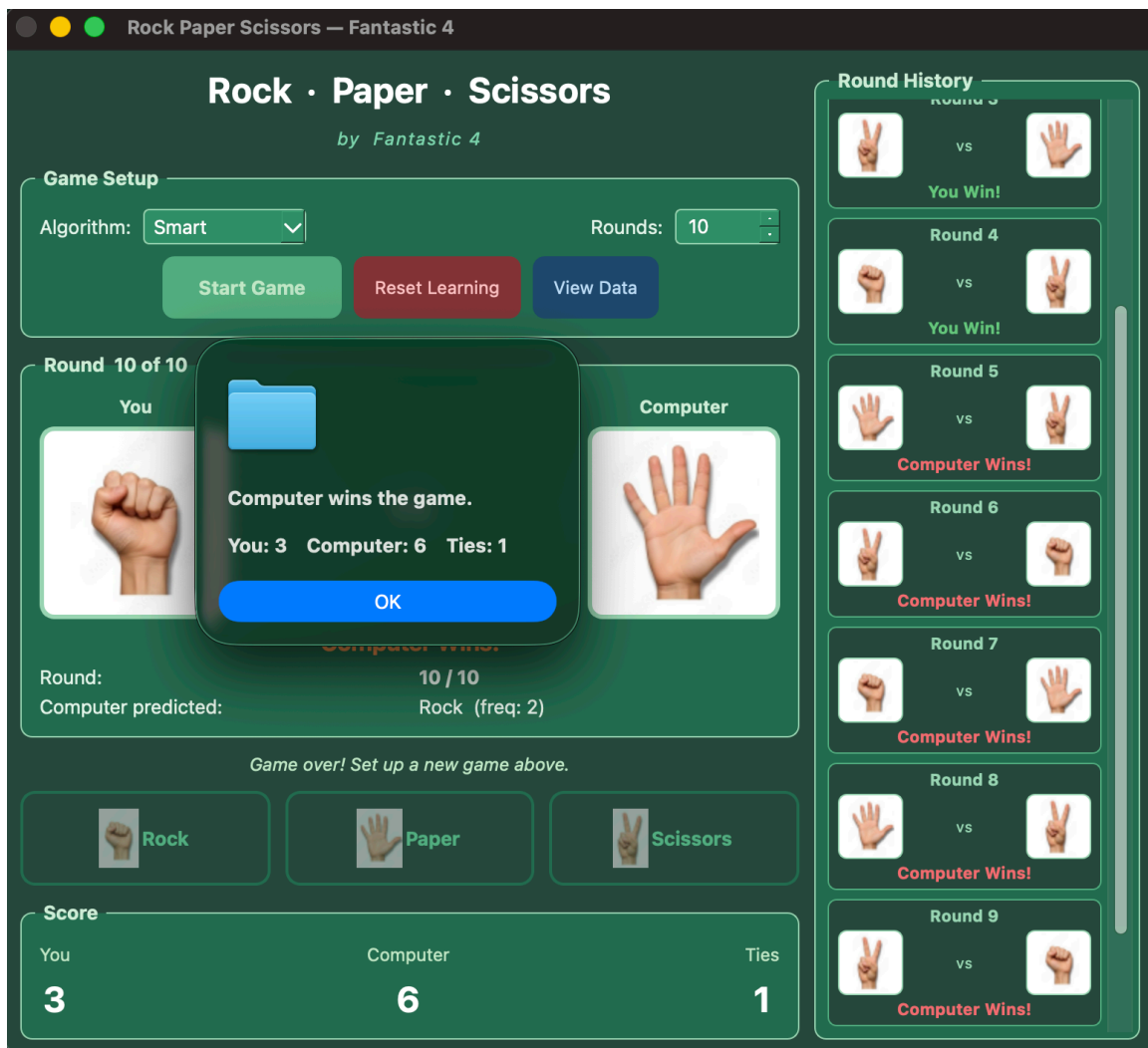


Suggested caption: Figure 1. Qt Designer layout with setup controls, active-round panel, score panel, and scrollable history panel.

Figure 2. Application During Gameplay

Place this image file at:

- Assignment7/report-assets/fig2_runtime_gameplay.png



Suggested caption: Figure 2. Runtime GUI showing current round, predicted human choice, selected moves, round outcome, and cumulative score.

10. Mapping to Assignment Criteria

10.1 Functional criteria coverage

The implementation satisfies rubric features for algorithm selection, round-count configuration, per-round move input, round-level state display, prediction visibility, winner display, and cumulative score tracking.

10.2 Design criteria coverage

Code design quality is supported by clear class boundaries, strategy interchangeability, UI-independent game logic, and mode-specific presentation adapters.

10.3 Dual-mode requirement

A single executable supports both console and GUI modes through command-line mode selection and shared engine semantics.

11. Conclusion

This project demonstrates a successful transition from a console-only implementation to a dual-mode software system that preserves architectural coherence. The key result is not only a functional GUI, but a maintainable design where core logic is reusable across presentation contexts. Inversion of control in GUI mode was handled through Qt event dispatch and signal-slot callbacks, while business logic remained encapsulated in reusable engine and strategy abstractions. The final system meets assignment requirements, supports extensibility, and provides a foundation for future algorithmic and interface enhancements.